

Universidade Federal de Viçosa
Campus
Rio Paranaíba

Leonardo Padovani Manzato – 9862

Análise de casos: Insertion Sort

Rio Paranaíba – MG

2026

Universidade Federal de Viçosa

Campus

Rio Paranaíba

Leonardo Padovani Manzato – 9862

Análise de casos: Insertion Sort

Trabalho apresentado para obtenção de créditos na disciplina SIN213 – Projeto de Algoritmo da Universidade Federal de Viçosa – Campus de Rio Paranaíba, ministrada pelo Professor Pedro Moisés de Souza.

Rio Paranaíba – MG

2026

1 RESUMO

Os algoritmos de ordenação apresentam grande importância no estudo teórico de algoritmos, bem como em aplicações rotineiras e práticas do cotidiano. Diante dessa grande importância, ao longo dos anos surgiram diversos algoritmos de ordenação na literatura, nos quais são analisados diferentes casos de execução. Sendo assim, torna-se necessário estudar o comportamento desses algoritmos, bem como suas complexidades, para indicar qual é o mais adequado para cada tipo de problema.

Entre os algoritmos de ordenação existentes, destaca-se o *Insertion Sort*, ou ordenação por inserção, que organiza os elementos de um vetor de maneira progressiva. Seu funcionamento consiste em percorrer os elementos e inserir cada um deles na posição correta dentro da parte do vetor que já se encontra ordenada. Por apresentar uma implementação simples e intuitiva, esse algoritmo pode ser eficiente em vetores pequenos ou parcialmente ordenados.

Neste trabalho, será apresentado o funcionamento do algoritmo *Insertion Sort*, incluindo sua implementação, um teste de mesa manual e a análise de sua complexidade nos melhores, médios e piores casos. Também será analisado seu comportamento por meio de experimentos com entradas crescentes, decrescentes e aleatórias, relacionando os resultados práticos com a análise teórica de complexidade.

Sumário

1	RESUMO	1
2	INTRODUÇÃO	3
3	ALGORITMOS	4
3.1	Insertion Sort	4
4	ANÁLISE DE COMPLEXIDADE	5
4.1	Insertion Sort	5
4.1.1	Melhor caso – $\Theta(n)$	5
4.1.2	Pior caso – $\Theta(n^2)$	5
4.1.3	Caso médio – $\Theta(n^2)$	6
4.1.4	Resumo das complexidades	7
5	TABELA E GRÁFICO	8
5.1	Insertion Sort	8
6	ATIVIDADES MANUSCRITAS	10
7	CONCLUSÃO	12
8	REFERÊNCIAS BIBLIOGRÁFICAS	13
9	FUNÇÃO PARA CALCULAR O TEMPO	14

2 INTRODUÇÃO

Em várias situações do nosso dia a dia, deparamo-nos com a necessidade de trabalhar com dados e informações devidamente ordenados, como, por exemplo, ao procurar um contato na lista telefônica. Imagine como seria difícil localizar um nome caso os contatos não estivessem organizados em ordem alfabética. Dessa forma, não é difícil perceber que as atividades que envolvem algum método de ordenação estão muito presentes na computação.

Os algoritmos de ordenação são responsáveis por organizar um conjunto de elementos de acordo com determinado critério, como ordem crescente, decrescente ou alfabética. Essa organização facilita a busca, a visualização e o processamento dos dados, contribuindo para a eficiência de diversas aplicações computacionais.

Existem diferentes algoritmos de ordenação, cada um apresentando características, vantagens e limitações próprias. Entre eles, encontra-se o algoritmo *Insertion Sort*, também conhecido como ordenação por inserção. Seu funcionamento baseia-se na construção progressiva de uma parte ordenada do vetor, inserindo cada novo elemento em sua posição correta.

O *Insertion Sort* apresenta uma implementação simples e de fácil compreensão, sendo especialmente útil para vetores pequenos ou parcialmente ordenados. Entretanto, seu custo de execução pode crescer quadraticamente em situações nas quais os elementos estão desordenados. Por esse motivo, a análise de sua complexidade é importante para compreender seu desempenho e determinar em quais situações sua utilização é mais adequada.

Neste trabalho, será apresentado o funcionamento do algoritmo *Insertion Sort*, juntamente com sua implementação, teste de mesa e análise de complexidade. Serão considerados o melhor caso, o caso médio e o pior caso, buscando compreender o comportamento do algoritmo e a quantidade de operações realizadas durante o processo de ordenação.

3 ALGORITMOS

3.1 Insertion Sort

O Insertion Sort é um algoritmo de ordenação baseado na comparação e inserção dos elementos em suas posições corretas. Seu funcionamento é semelhante ao ato de organizar cartas de baralho: os elementos são analisados um por vez e cada novo elemento é inserido na posição adequada da parte que já está ordenada.

Inicialmente, considera-se que o primeiro elemento do vetor já está ordenado. Em seguida, o algoritmo seleciona o próximo elemento, chamado de chave, e compara seu valor com os elementos anteriores. Enquanto esses elementos forem maiores que a chave, eles são deslocados uma posição para a direita. Ao final, a chave é inserida na posição correta.

Listing 1: Implementação do Insertion Sort

```
1 void insertionSort(int* vetor, int tamanho) {
2     for (int i = 1; i < tamanho; i++) {
3
4         int chave = vetor[i];
5         int j = i - 1;
6
7         while (j >= 0 && vetor[j] > chave) {
8             vetor[j + 1] = vetor[j];
9             j--;
10        }
11
12        vetor[j + 1] = chave;
13    }
14 }
```

4 ANÁLISE DE COMPLEXIDADE

4.1 Insertion Sort

O algoritmo Insertion Sort apresenta diferentes comportamentos de desempenho dependendo da organização inicial dos elementos do vetor. Sua complexidade de tempo é analisada considerando o melhor caso, o caso médio e o pior caso.

4.1.1 Melhor caso – $\Theta(n)$

O melhor caso ocorre quando o vetor já está ordenado de forma crescente. Nessa situação, para cada elemento analisado, o algoritmo realiza apenas uma comparação com o elemento anterior e verifica que não é necessário deslocar nenhum elemento.

Considerando um vetor com n elementos, o laço externo é executado $n - 1$ vezes. Em cada iteração, é realizada uma quantidade constante de operações, como atribuições e comparações.

Assim, o número total de operações é proporcional a:

$$T(n) = c(n - 1) + d \quad (1)$$

em que c e d são constantes. Desconsiderando os termos constantes e os de menor crescimento, temos:

$$T(n) = \Theta(n) \quad (2)$$

Portanto, o melhor caso do Insertion Sort possui complexidade linear.

4.1.2 Pior caso – $\Theta(n^2)$

O pior caso ocorre quando o vetor está ordenado de forma decrescente e deseja-se ordená-lo de forma crescente. Nesse cenário, cada novo elemento precisa ser comparado com todos os elementos anteriores e deslocado até o início da parte já ordenada.

A quantidade total de deslocamentos pode ser representada pela soma:

$$T(n) = 1 + 2 + 3 + \cdots + (n - 1) \quad (3)$$

Essa soma corresponde à soma dos termos de uma progressão aritmética:

$$T(n) = \frac{n(n-1)}{2} \quad (4)$$

Desenvolvendo a expressão:

$$T(n) = \frac{n^2 - n}{2} \quad (5)$$

O termo de maior crescimento é n^2 . Portanto:

$$T(n) = \Theta(n^2) \quad (6)$$

Assim, o pior caso do Insertion Sort apresenta complexidade quadrática.

4.1.3 Caso médio – $\Theta(n^2)$

O caso médio ocorre quando os elementos estão distribuídos aleatoriamente. Nessa situação, o algoritmo realiza, em média, aproximadamente metade dos deslocamentos possíveis para cada elemento inserido.

Temos:

$$T(n) \approx \frac{1}{2} (1 + 2 + 3 + \dots + (n-1)) \quad (7)$$

Aplicando a fórmula da soma:

$$T(n) \approx \frac{1}{2} \cdot \frac{n(n-1)}{2} \quad (8)$$

$$T(n) \approx \frac{n^2 - n}{4} \quad (9)$$

Como o termo dominante é n^2 , a complexidade assintótica é:

$$T(n) = \Theta(n^2) \quad (10)$$

Portanto, no caso médio, o Insertion Sort também apresenta complexidade quadrática.

4.1.4 Resumo das complexidades

Tabela 1: Complexidades do algoritmo Insertion Sort

Caso	Complexidade de tempo
Melhor caso	$\Theta(n)$
Caso médio	$\Theta(n^2)$
Pior caso	$\Theta(n^2)$

Conclui-se que o Insertion Sort é eficiente para vetores pequenos ou parcialmente ordenados, mas apresenta desempenho inferior para grandes conjuntos de dados desordenados.

5 TABELA E GRÁFICO

5.1 Insertion Sort

Resultados dos experimentos — Insertion Sort

Tamanho (n)	Crescente (s)	Decrescente (s)	Aleatória (s)
10.0	0.0	0.0	0.0
100.0	0.0	3e-06	3e-06
1000.0	1e-06	0.000239	0.000166
10000.0	7e-06	0.048499	0.0233
100000.0	6.8e-05	4.964015	2.45468
1000000.0	0.001696	535.4135	272.5905

Figura 1: Tabela de tempo de execução do algoritmo Insertion Sort

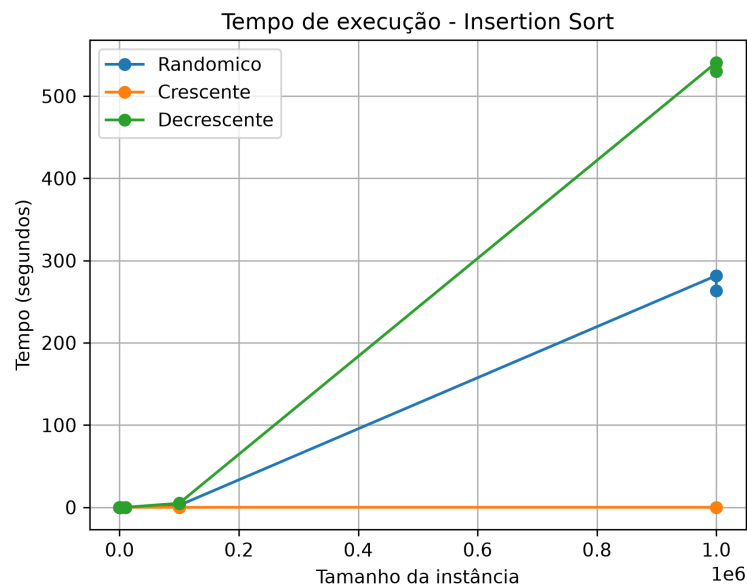


Figura 2: Gráfico de tempo de execução do algoritmo Insertion Sort

Os resultados obtidos nos experimentos estão apresentados na tabela e no gráfico. Observe-se que o algoritmo Insertion Sort apresenta melhor desempenho para instâncias ordenadas de forma crescente, uma vez que, nesse caso, praticamente não são necessárias movimentações dos elementos.

Em contrapartida, as instâncias ordenadas de forma decrescente apresentam os maiores tempos de execução. Isso ocorre porque cada novo elemento precisa ser comparado e deslocado por uma grande quantidade de posições, caracterizando o pior caso do algoritmo, cuja complexidade é $\Theta(n^2)$.

As instâncias aleatórias apresentam desempenho intermediário, pois a quantidade de deslocamentos depende da disposição inicial dos elementos.

A partir dos dados, também é possível perceber que o aumento do tamanho da entrada provoca um crescimento significativo no tempo de execução, principalmente nos casos aleatório e decrescente. Para a instância de tamanho $n = 1.000.000$, o tempo registrado foi aproximadamente 272,59 segundos para a entrada aleatória e 535,41 segundos para a entrada decrescente. Já para a entrada crescente, o tempo foi de aproximadamente 0,001696 segundos.

Esses resultados confirmam o comportamento teórico esperado para o algoritmo Insertion Sort.

6 ATIVIDADES MANUSCRITAS

D S T Q Q S S
D L M M J V S

Teste de mesa manual - Insertion Sort

$V = [7, 3, 9, 2, 5]$

- $n = 5$, logo, será executado $n-1$ vezes, ou seja, 4 vezes
- Chaves $\rightarrow 3, 9, 2, 5$

Tabela: qtd. de testes de condição

Iteração	Chave	Vet. após	t_i	Deslocamentos
$i=2$	3	[3, 7, 9, 2, 5]	2	1
$i=3$	9	[3, 7, 9, 2, 5]	1	0
$i=4$	2	[2, 3, 7, 9, 5]	4	3
$i=5$	5	[2, 3, 5, 7, 9]	3	2

Soma de execuções do "while":

$$\sum t_i = 2 + 1 + 4 + 3 = 10$$

Soma dos deslocamentos:

$$\sum (t_i - 1) = 1 + 0 + 3 + 2 = 6$$

Tabela de custos:

Custo	Operação	Vezez	$O + (n)$ pode ser dado por:
C1	inicializar 'i'	1	
C2	teste do 'for'	$n=5$	$T(n) = c_1 + 5c_2 + 4c_3 + 4c_4 +$
C3	atribuir 'chave'	$n-1=4$	$10c_5 + 6c_6 + 6c_7 + 4c_8$
C4	atribuir 'j'	$n-1=4$	$T(n) = (c_1 + c_2 + 10c_5 + 6c_6 + 6c_7 + 4c_8) \cdot n +$
C5	teste do 'while'	$\sum t_i = 10$	Obs: m. melhor caso, c6 e c7
C6	deslocamento ' $A[j+1] = A[j]$ '	$\sum (t_i - 1) = 6$	e 0 é c5 é $(n-1)$ pois há
C7	decremento 'j'	$\sum (t_i - 1) = 6$	apenas 1 teste do 'while'.
C8	insere 'chave'	$n-1=4$	

Nota: + custo C6 e C7 como 0
m. melhor caso é c5 como $(n-1)$

Jandaia

Figura 3: Teste de mesa manual do algoritmo Insertion Sort

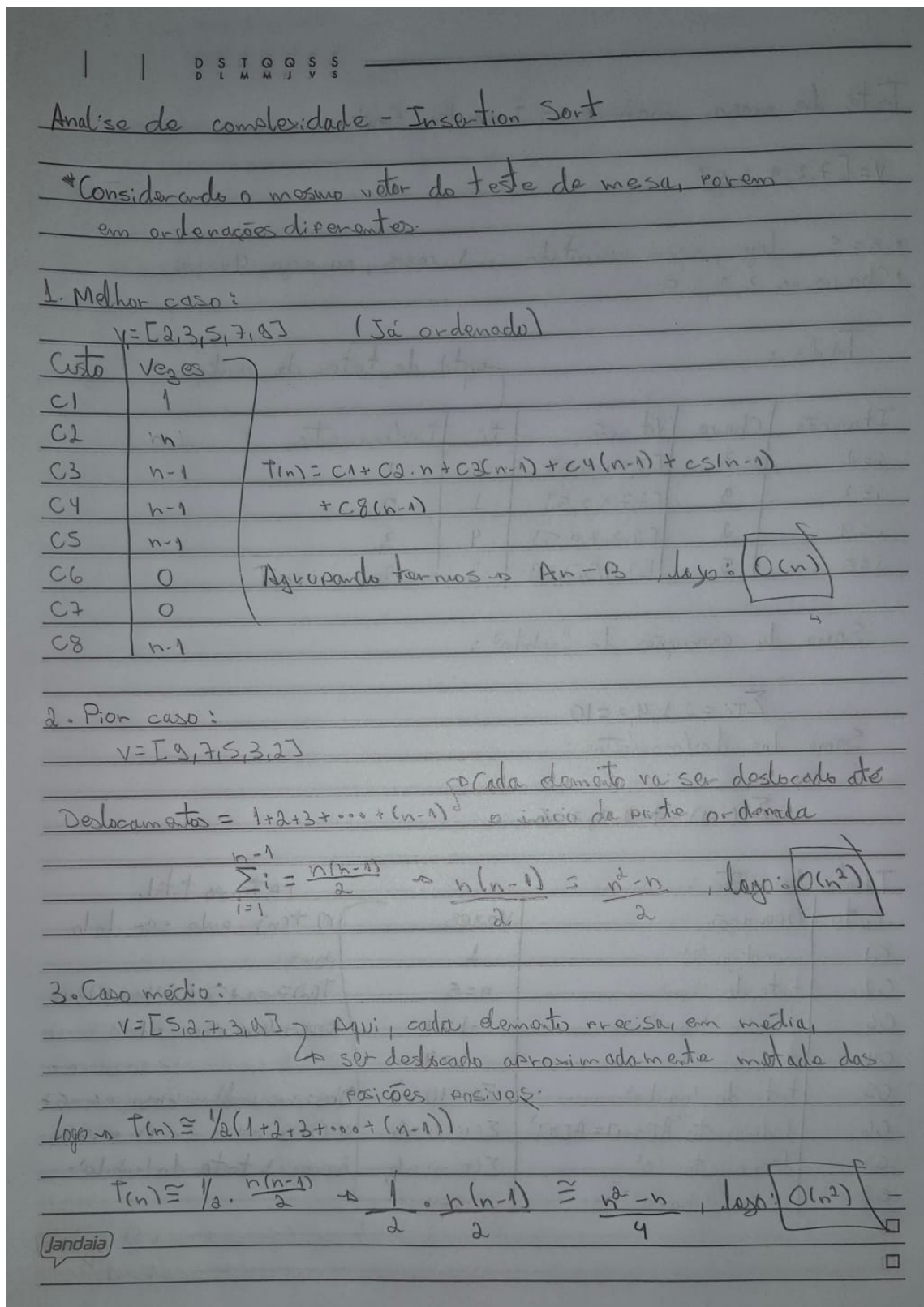


Figura 4: Análise de complexidade do algoritmo Insertion Sort

7 CONCLUSÃO

A realização deste trabalho permitiu compreender o funcionamento do algoritmo Insertion Sort e analisar seu comportamento em diferentes tipos de entrada. Por meio da implementação, do teste de mesa e dos experimentos práticos, foi possível observar a relação entre a organização inicial dos dados e o tempo necessário para realizar a ordenação.

A análise teórica mostrou que o melhor caso ocorre quando o vetor já está ordenado de forma crescente, apresentando complexidade $\Theta(n)$. Nesse cenário, são realizadas poucas comparações e praticamente nenhum deslocamento. Já o pior caso ocorre quando o vetor está ordenado de forma decrescente, exigindo maior quantidade de comparações e deslocamentos e apresentando complexidade $\Theta(n^2)$.

Os resultados experimentais confirmaram o comportamento previsto pela análise assintótica. As entradas crescentes apresentaram os menores tempos de execução, enquanto as entradas decrescentes apresentaram os maiores tempos. As entradas aleatórias apresentaram comportamento intermediário, como esperado para o caso médio.

Dessa forma, conclui-se que o Insertion Sort é uma alternativa adequada para vetores pequenos ou parcialmente ordenados, devido à sua simplicidade e facilidade de implementação. Entretanto, não é indicado para grandes volumes de dados desordenados, nos quais algoritmos com melhor complexidade assintótica podem apresentar desempenho superior.

8 REFERÊNCIAS BIBLIOGRÁFICAS

Referências

- [1] DEVFURIA. *Introdução ao algoritmo de ordenação Insertion Sort*. Disponível em: <http://devfuria.com.br/logica-de-programacao/introducao-ao-algoritmo-de-ordenacao-insertion-sort/>. Acesso em: 14 set. 2026.
- [2] cplusplus.com. *rand - C++ Reference*. Disponível em: <https://cplusplus.com/reference/cstdlib/rand/>. Acesso em: 14 set. 2026.
- [3] GURU99. *C++ Manipulação de arquivos: como abrir, gravar, ler e fechar arquivos em C++*. Disponível em: <https://www.guru99.com/pt/cpp-file-read-write-open.html>. Acesso em: 14 set. 2026.
- [4] NGHIA TU. *8 maneiras de medir o tempo de execução em C / C++*. Disponível em: <https://ichi.nghiatu.com/pt/8-maneiras-de-medir-o-tempo-de-execucao-em-c-c-79591185633529>. Acesso em: 14 set. 2026.
- [5] STACK OVERFLOW EM PORTUGUÊS. *Como passar uma função como parâmetro em C?*. Disponível em: <https://pt.stackoverflow.com/questions/2983/como-passar-uma-fun%C3%A7%C3%A3o-como-par%C3%A2metro-em-c>. Acesso em: 14 set. 2026.
- [6] OPENAI. *ChatGPT*: ferramenta de inteligência artificial utilizada como auxílio na elaboração e revisão dos scripts de geração de gráficos e tabelas. 2026.

9 FUNÇÃO PARA CALCULAR O TEMPO

Listing 2: Função utilizada para medir o tempo de execução

```
1 double calcularTempo(void (*algoritmo)(int*, int), int* vetor, int
  tamanho) {
2     // salvar tempo do inicio
3     auto inicio = chrono::steady_clock::now();
4
5     // executar o algoritmo
6     algoritmo(vetor, tamanho);
7
8     // salvar o tempo do fim da execucao
9     auto fim = chrono::steady_clock::now();
10
11    // calcular duracao
12    chrono::duration<double> tempo = fim - inicio;
13
14    return tempo.count();
15 }
```

A função apresentada no Código 2 é responsável por medir o tempo de execução do algoritmo. Para isso, registra o instante inicial, executa o algoritmo, registra o instante final e calcula a diferença entre os dois instantes.

O valor retornado pela função representa o tempo de execução em segundos.